

C Strings: Input Methods and Logical Programming Exercises

Dr. Mohammad Maksood Akhter
Assistant Professor
Department of Computer Science and Engineering

I. INTRODUCTION TO STRINGS IN C

A string in C is a sequence of characters terminated by a special character called the null character, represented by `\0`.

For example:

```
char str[] = "Hello";
```

Internally, the string is stored as:

H e l l o \0

Therefore, a string containing n visible characters requires at least $n + 1$ character locations.

Every valid C string must end with the null character `\0`. The null character tells string-processing functions where the string ends.

II. METHODS FOR TAKING STRING INPUT

There are several ways to take string input in C. The three important methods considered in this section are:

- 1) `scanf()`
- 2) `fgets()`
- 3) `getchar()`

A. Input Method 1: `scanf()`

The `scanf()` function can be used with the `%s` format specifier to read a string.

```
1  #include <stdio.h>
2
3  int main() {
4      char str[50];
5
6      printf("Enter a word: ");
7      scanf("%s", str);
8
9      printf("String = %s\n", str);
10
11     return 0;
12 }
```

When using `%s`, the array name is passed without the `&` operator because the array name represents the address of its first element.

For example, if the input is:

```
Computer
```

the complete word is stored.

However, if the input is:

```
Computer Science
```

only Computer is read because `scanf("%s", ...)` stops at whitespace.

`scanf("%s", str)` is suitable when the input contains a single word. It does not read spaces as part of the string.

B. Input Method 2: `fgets()`

The `fgets()` function can be used to read an entire line, including spaces.

Syntax:

```
1 fgets(string, size, stdin);
```

Example:

```
1 #include <stdio.h>
2
3 int main() {
4     char str[50];
5
6     printf("Enter a sentence: ");
7     fgets(str, sizeof(str), stdin);
8
9     printf("String = %s", str);
10
11     return 0;
12 }
```

If the user enters:

```
1 Computer Science
```

the complete input line is stored in the array.

`fgets()` limits the number of characters read according to the supplied buffer size. It is therefore preferable to `gets()` for safe line input.

1) *The Newline Character with `fgets()`*: When the user presses Enter, `fgets()` may store the newline character `\n` if there is sufficient space.

For example:

```
1 char str[20];
2
3 fgets(str, sizeof(str), stdin);
```

The stored string may contain:

```
1 Hello\n\0
```

The newline can be removed as follows:

```
1 #include <string.h>
2
3 str[strcspn(str, "\n")] = '\0';
```

C. Input Method 3: `getchar()`

The `getchar()` function reads one character from the input stream at a time.

Example:

```
1 #include <stdio.h>
2
3 int main() {
4     char ch;
5
6     printf("Enter a character: ");
7     ch = getchar();
8
9     printf("You entered: %c\n", ch);
10
11     return 0;
12 }
```

`getchar()` returns the next character available from `stdin`.

1) *Reading a String Using getchar():* A complete string can be constructed by repeatedly calling `getchar()`.

```

1      #include <stdio.h>
2
3      int main() {
4          char str[100];
5          int i = 0;
6          char ch;
7
8          printf("Enter a string: ");
9
10         while ((ch = getchar()) != '\n') {
11             str[i] = ch;
12             i++;
13         }
14
15         str[i] = '\0';
16
17         printf("String = %s\n", str);
18
19         return 0;
20     }

```

The loop continues until the user presses Enter. The statement `str[i] = '\0'` explicitly terminates the string.

III. COMPARISON OF STRING INPUT METHODS

Method	Spaces	Input	Typical Use
<code>scanf("%s")</code>	No	Word	Single-word input
<code>fgets()</code>	Yes	Complete line	Line/string input
<code>getchar()</code>	Yes	Character	Character-level processing

Do not use `gets()` in modern C programs. It does not provide a way to specify the size of the destination array and can cause buffer overflow.

IV. LOGICAL PROGRAMMING EXERCISES ON STRINGS

The following exercises focus on string manipulation and logical problem solving rather than simply using predefined string functions.

A. Lab Question 1: Count Vowels and Consonants

Write a C program to input a string and count the number of vowels and consonants present in the string. Ignore spaces, digits, and special characters. Do not use any library function for counting vowels.

```

1      #include <stdio.h>
2
3      int main() {
4          char str[100];
5          int i, vowels = 0, consonants = 0;
6
7          printf("Enter a string: ");
8          fgets(str, sizeof(str), stdin);
9
10         for (i = 0; str[i] != '\0'; i++) {
11
12             if ((str[i] >= 'A' && str[i] <= 'Z') ||
13                 (str[i] >= 'a' && str[i] <= 'z')) {
14
15                 if (str[i] == 'a' || str[i] == 'e' ||
16                     str[i] == 'i' || str[i] == 'o' ||
17                     str[i] == 'u' || str[i] == 'A' ||
18                     str[i] == 'E' || str[i] == 'I' ||
19                     str[i] == 'O' || str[i] == 'U')
20                     vowels++;
21                 else
22                     consonants++;
23             }
24         }
25     }

```

```

24     }
25
26     printf("Vowels = %d\n", vowels);
27     printf("Consonants = %d\n", consonants);
28
29     return 0;
30 }

```

B. Lab Question 2: Reverse a String

Write a C program to input a string and print its reverse.
Do not use any predefined reverse-string function.

```

1     #include <stdio.h>
2
3     int main() {
4         char str[100];
5         int i, length = 0;
6
7         printf("Enter a string: ");
8         fgets(str, sizeof(str), stdin);
9
10        while (str[length] != '\0' &&
11               str[length] != '\n')
12            length++;
13
14        printf("Reverse = ");
15
16        for (i = length - 1; i >= 0; i--)
17            printf("%c", str[i]);
18
19        printf("\n");
20
21        return 0;
22    }

```

First determine the logical length of the string and then traverse the characters from the last position toward the first position.

C. Lab Question 3: Check Whether a String is Palindrome

Write a C program to determine whether an input string is a palindrome.
A palindrome reads the same from left to right and right to left.
Examples: madam, level, radar.

```

1     #include <stdio.h>
2
3     int main() {
4         char str[100];
5         int i, j, length = 0;
6         int palindrome = 1;
7
8         printf("Enter a string: ");
9         fgets(str, sizeof(str), stdin);
10
11        while (str[length] != '\0' && str[length] != '\n')
12            length++;
13
14        i = 0;
15        j = length - 1;
16
17        while (i < j) {
18            if (str[i] != str[j]) {
19                palindrome = 0;
20                break;
21            }
22        }

```

```

23         i++;
24         j--;
25     }
26
27     if (palindrome)
28         printf("Palindrome\n");
29     else
30         printf("Not a palindrome\n");
31
32     return 0;
33 }

```

D. Lab Question 4: Count Frequency of a Character

Write a C program to input a string and a character. Find how many times the given character occurs in the string. The comparison should be case-sensitive.

```

1     #include <stdio.h>
2
3     int main() {
4         char str[100];
5         char ch;
6         int i, count = 0;
7
8         printf("Enter a string: ");
9         fgets(str, sizeof(str), stdin);
10
11        printf("Enter character to search: ");
12        scanf(" %c", &ch);
13
14        for (i = 0; str[i] != '\0'; i++) {
15            if (str[i] == ch)
16                count++;
17        }
18
19        printf("Frequency = %d\n", count);
20
21        return 0;
22    }

```

Notice the space before %c in scanf(" %c", &ch). It tells scanf() to skip leading whitespace characters.

E. Lab Question 5: Count Words in a String

Write a C program to count the number of words in a sentence. Assume that words are separated by one or more spaces.

```

1     #include <stdio.h>
2
3     int main() {
4         char str[200];
5         int i, words = 0;
6         int inWord = 0;
7
8         printf("Enter a sentence: ");
9         fgets(str, sizeof(str), stdin);
10
11        for (i = 0; str[i] != '\0'; i++) {
12
13            if (str[i] != ' ' && str[i] != '\n' && str[i] != '\t') {
14
15                if (inWord == 0) {
16                    words++;
17                    inWord = 1;
18                }
19            }
20            else {

```

```

21         inWord = 0;
22     }
23 }
24
25 printf("Number of words = %d\n", words);
26
27 return 0;
28 }

```

The variable `inWord` represents the current state: 0 means outside a word and 1 means currently inside a word.

F. Lab Question 6: Remove All Spaces

Write a C program to input a string and remove all spaces from it.

Example:

Hello World in C → HelloWorldinC

Do not use a predefined string replacement function.

```

1  #include <stdio.h>
2
3  int main() {
4      char str[100];
5      int i, j = 0;
6
7      printf("Enter a string: ");
8      fgets(str, sizeof(str), stdin);
9
10     for (i = 0; str[i] != '\0'; i++) {
11         if (str[i] != ' ' && str[i] != '\n' && str[i] != '\t') {
12             str[j] = str[i];
13             j++;
14         }
15     }
16
17     str[j] = '\0';
18
19     printf("Result = %s\n", str);
20
21     return 0;
22 }
23
24

```

G. Lab Question 7: Compare Two Strings Without strcmp()

Write a C program to input two strings and determine whether they are equal.

Do not use the `strcmp()` library function.

```

1  #include <stdio.h>
2
3  int main() {
4      char str1[100], str2[100];
5      int i = 0;
6      int equal = 1;
7
8      printf("Enter first string: ");
9      fgets(str1, sizeof(str1), stdin);
10
11     printf("Enter second string: ");
12     fgets(str2, sizeof(str2), stdin);
13
14     while (str1[i] != '\0' || str2[i] != '\0') {
15         if (str1[i] == '\n' && str2[i] == '\n')
16             break;
17         if (str1[i] != str2[i]) {
18             equal = 0;
19             break;
20         }
21         i++;
22     }
23
24     if (equal) {
25         printf("Strings are equal.\n");
26     } else {
27         printf("Strings are not equal.\n");
28     }
29
30     return 0;
31 }

```

```

17         break;
18
19         if (str1[i] != str2[i]) {
20             equal = 0;
21             break;
22         }
23
24         i++;
25     }
26
27     if (equal)
28         printf("Strings are equal\n");
29     else
30         printf("Strings are not equal\n");
31
32     return 0;
33 }

```

H. Lab Question 8: Find the Length Without strlen()

Write a C program to find the length of a string without using the `strlen()` function. Do not count the newline character inserted by `fgets()`.

```

1     #include <stdio.h>
2
3     int main() {
4         char str[100];
5         int i = 0;
6
7         printf("Enter a string: ");
8         fgets(str, sizeof(str), stdin);
9
10        while (str[i] != '\0' &&
11               str[i] != '\n') {
12            i++;
13        }
14
15        printf("Length = %d\n", i);
16
17        return 0;
18    }

```

I. Lab Question 9: Convert Lowercase to Uppercase

Write a C program to convert all lowercase English letters in a string into uppercase letters. Do not use `strupr()` or other predefined conversion functions.

```

1     #include <stdio.h>
2
3     int main() {
4         char str[100];
5         int i;
6
7         printf("Enter a string: ");
8         fgets(str, sizeof(str), stdin);
9
10        for (i = 0; str[i] != '\0'; i++) {
11
12            if (str[i] >= 'a' && str[i] <= 'z')
13                str[i] = str[i] - 'a' + 'A';
14        }
15
16        printf("Uppercase = %s", str);
17
18        return 0;
19    }

```

J. Lab Question 10: Find the First Non-Repeated Character

Write a C program to find the first character in a string that appears exactly once.

Example:

SWISS → W

Do not use predefined frequency-counting functions.

```

1      #include <stdio.h>
2
3      int main() {
4          char str[100];
5          int i, j, count;
6          int found = 0;
7
8          printf("Enter a string: ");
9          fgets(str, sizeof(str), stdin);
10
11         for (i = 0; str[i] != '\0' && str[i] != '\n'; i++) {
12             count = 0;
13
14             for (j = 0; str[j] != '\0' && str[j] != '\n'; j++) {
15                 if (str[i] == str[j])
16                     count++;
17             }
18
19             if (count == 1) {
20                 printf("First non-repeated character = %c\n",
21                     str[i]);
22                 found = 1;
23                 break;
24             }
25         }
26
27         if (!found)
28             printf("No non-repeated character found.\n");
29
30         return 0;
31     }
32
33 
```

V. PRACTICE QUESTIONS WITH SOLUTIONS

A. Question 1: Count Uppercase, Lowercase, Digits, and Special Characters

Write a C program to count uppercase letters, lowercase letters, digits, and special characters in a string.

```

1      #include <stdio.h>
2
3      int main() {
4          char str[200];
5          int i;
6          int upper = 0, lower = 0;
7          int digits = 0, special = 0;
8
9          printf("Enter a string: ");
10         fgets(str, sizeof(str), stdin);
11
12         for (i = 0; str[i] != '\0'; i++) {
13             if (str[i] >= 'A' && str[i] <= 'Z')
14                 upper++;
15
16             else if (str[i] >= 'a' && str[i] <= 'z')
17                 lower++;
18
19             else if (str[i] >= '0' && str[i] <= '9')
20                 digits++;
21
22             else
23                 special++;
24         }
25
26         printf("Uppercase: %d, Lowercase: %d, Digits: %d, Special: %d\n",
27             upper, lower, digits, special);
28     }
29 
```

```

23         else if (str[i] != '\n')
24             special++;
25     }
26
27     printf("Uppercase = %d\n", upper);
28     printf("Lowercase = %d\n", lower);
29     printf("Digits = %d\n", digits);
30     printf("Special characters = %d\n", special);
31
32     return 0;
33 }

```

The ASCII ranges 'A'--'Z', 'a'--'z', and '0'--'9' can be used to classify characters without additional library functions.

B. Question 2: Count Occurrences of Each Vowel

Write a C program to count the number of occurrences of each vowel (a, e, i, o, and u) in a string. The program should treat uppercase and lowercase vowels as the same.

```

1  #include <stdio.h>
2
3  int main() {
4      char str[200];
5      int i;
6      int a = 0, e = 0, in = 0;
7      int o = 0, u = 0;
8
9      printf("Enter a string: ");
10     fgets(str, sizeof(str), stdin);
11
12     for (i = 0; str[i] != '\0'; i++) {
13
14         switch (str[i]) {
15
16             case 'a':
17             case 'A':
18                 a++;
19                 break;
20
21             case 'e':
22             case 'E':
23                 e++;
24                 break;
25
26             case 'i':
27             case 'I':
28                 in++;
29                 break;
30
31             case 'o':
32             case 'O':
33                 o++;
34                 break;
35
36             case 'u':
37             case 'U':
38                 u++;
39                 break;
40         }
41     }
42
43     printf("a = %d\n", a);
44     printf("e = %d\n", e);
45     printf("i = %d\n", in);
46     printf("o = %d\n", o);
47     printf("u = %d\n", u);
48
49     return 0;
50 }

```

A switch statement is useful when the program must perform different actions for a fixed set of character values.

C. Question 3: Reverse a String Using getchar()

Write a C program to read a string using `getchar()` and print the string in reverse order. Do not use `strlen()` or any predefined reverse-string function.

```

1  #include <stdio.h>
2
3  int main() {
4      char str[200];
5      int i = 0;
6      char ch;
7
8      printf("Enter a string: ");
9
10     while ((ch = getchar()) != '\n' && ch != EOF) {
11
12         if (i < 199) {
13             str[i] = ch;
14             i++;
15         }
16     }
17
18     str[i] = '\0';
19
20     printf("Reverse = ");
21
22     for (i = i - 1; i >= 0; i--)
23         printf("%c", str[i]);
24
25     printf("\n");
26
27     return 0;
28 }
```

The statement `str[i] = '\0'` is required to terminate the character array as a valid C string. The reverse operation then starts from the last stored character.

D. Question 4: Check Whether Two Strings Are Anagrams

Write a C program to determine whether two strings are anagrams. Two strings are anagrams if they contain the same characters with the same frequencies, regardless of their order. For example:

listen and silent

are anagrams.

Ignore spaces and consider uppercase and lowercase letters as different characters.

```

1  #include <stdio.h>
2
3  int main() {
4      char str1[200], str2[200];
5      int freq1[256] = {0};
6      int freq2[256] = {0};
7      int i;
8      int anagram = 1;
9
10     printf("Enter first string: ");
11     fgets(str1, sizeof(str1), stdin);
12
13     printf("Enter second string: ");
14     fgets(str2, sizeof(str2), stdin);
15 }
```

```

16     for (i = 0; str1[i] != '\0'; i++) {
17
18         if (str1[i] != ' ' && str1[i] != '\n' && str1[i] != '\t')
19             freq1[(unsigned char)str1[i]]++;
20     }
21
22     for (i = 0; str2[i] != '\0'; i++) {
23
24         if (str2[i] != ' ' &&
25             str2[i] != '\n' &&
26             str2[i] != '\t')
27             freq2[(unsigned char)str2[i]]++;
28     }
29
30     for (i = 0; i < 256; i++) {
31
32         if (freq1[i] != freq2[i]) {
33             anagram = 0;
34             break;
35         }
36     }
37
38     if (anagram)
39         printf("The strings are anagrams.\n");
40     else
41         printf("The strings are not anagrams.\n");
42
43     return 0;
44 }

```

Instead of sorting both strings, the program counts the frequency of every character. If all corresponding frequencies are equal, the strings are anagrams.

E. Question 5: Remove Duplicate Characters

Write a C program to remove duplicate characters from a string.
For example:

programming → progamin

The first occurrence of each character should be retained.

```

1  #include <stdio.h>
2
3  int main() {
4      char str[200];
5      int i, j;
6      int found;
7
8      printf("Enter a string: ");
9      fgets(str, sizeof(str), stdin);
10
11     for (i = 0; str[i] != '\0'; i++) {
12
13         if (str[i] == '\n')
14             break;
15
16         found = 0;
17
18         for (j = 0; j < i; j++) {
19
20             if (str[i] == str[j]) {
21                 found = 1;
22                 break;
23             }
24         }
25
26         if (!found)
27             printf("%c", str[i]);
28     }

```

```

29
30     printf("\n");
31
32     return 0;
33 }

```

For each character, check whether the same character has already appeared at an earlier position. If it has not appeared, print it.

F. Question 6: Find the Largest Word in a Sentence

Write a C program to find the largest word in a sentence.

Assume that words are separated by spaces.

Example:

C programming is interesting

The largest word is programming.

```

1  #include <stdio.h>
2
3  int main() {
4      char str[300];
5      char largest[100];
6      int i = 0, j;
7      int start = 0;
8      int currentLength = 0;
9      int largestLength = 0;
10
11     printf("Enter a sentence: ");
12     fgets(str, sizeof(str), stdin);
13
14     while (1) {
15
16         if (str[i] != ' ' &&
17             str[i] != '\n' &&
18             str[i] != '\0') {
19
20             currentLength++;
21         }
22         else {
23
24             if (currentLength > largestLength) {
25
26                 largestLength = currentLength;
27
28                 for (j = 0; j < currentLength; j++)
29                     largest[j] = str[start + j];
30
31                 largest[currentLength] = '\0';
32             }
33
34             currentLength = 0;
35             start = i + 1;
36         }
37
38         if (str[i] == '\0' ||
39             str[i] == '\n')
40             break;
41
42         i++;
43     }
44
45     printf("Largest word = %s\n", largest);
46     printf("Length = %d\n", largestLength);
47
48     return 0;
49 }

```

The program maintains the starting position and length of the current word. Whenever a longer word is found, it is copied into the `largest` array.

G. Question 7: Find the Smallest Word in a Sentence

Write a C program to find the smallest word in a sentence.

Assume that words are separated by spaces.

Example:

C programming is fun

The smallest word is C.

```

1  #include <stdio.h>
2
3  int main() {
4      char str[300];
5      char smallest[100];
6      int i = 0, j;
7      int start = 0;
8      int currentLength = 0;
9      int smallestLength = 1000;
10
11     printf("Enter a sentence: ");
12     fgets(str, sizeof(str), stdin);
13
14     while (1) {
15
16         if (str[i] != ' ' &&
17             str[i] != '\n' &&
18             str[i] != '\0') {
19
20             currentLength++;
21         }
22         else {
23
24             if (currentLength > 0 &&
25                 currentLength < smallestLength) {
26
27                 smallestLength = currentLength;
28
29                 for (j = 0; j < currentLength; j++)
30                     smallest[j] = str[start + j];
31
32                 smallest[currentLength] = '\0';
33             }
34
35             currentLength = 0;
36             start = i + 1;
37         }
38
39         if (str[i] == '\0' ||
40             str[i] == '\n')
41             break;
42
43         i++;
44     }
45
46     if (smallestLength == 1000)
47         printf("No word found.\n");
48     else {
49         printf("Smallest word = %s\n", smallest);
50         printf("Length = %d\n", smallestLength);
51     }
52
53     return 0;
54 }
```

H. Question 8: Replace a Character in a String

Write a C program to replace every occurrence of a given character with another character.
For example:

banana, a → o

produces:

bonono

```

1  #include <stdio.h>
2
3  int main() {
4      char str[200];
5      char oldChar, newChar;
6      int i;
7
8      printf("Enter a string: ");
9      fgets(str, sizeof(str), stdin);
10
11     printf("Enter character to replace: ");
12     scanf(" %c", &oldChar);
13
14     printf("Enter replacement character: ");
15     scanf(" %c", &newChar);
16
17     for (i = 0; str[i] != '\0'; i++) {
18
19         if (str[i] == oldChar)
20             str[i] = newChar;
21     }
22
23     printf("Result = %s", str);
24
25     return 0;
26 }
```

Since strings are stored as character arrays, individual characters can be modified directly using their array indices.

I. Question 9: Count the Frequency of Every Character

Write a C program to count the frequency of every character present in a string.
Ignore spaces and the newline character.

```

1  #include <stdio.h>
2
3  int main() {
4      char str[200];
5      int frequency[256] = {0};
6      int i;
7
8      printf("Enter a string: ");
9      fgets(str, sizeof(str), stdin);
10
11     for (i = 0; str[i] != '\0'; i++) {
12
13         if (str[i] != ' ' &&
14             str[i] != '\n' &&
15             str[i] != '\t') {
16
17             frequency[(unsigned char)str[i]]++;
18         }
19     }
20
21     printf("\nCharacter frequencies:\n");
22
23     for (i = 0; i < 256; i++) {
```

```

24         if (frequency[i] > 0)
25             printf("%c : %d\n", i, frequency[i]);
26     }
27
28     return 0;
29 }
30

```

The array `frequency[256]` acts as a frequency table. Each character is mapped to its character-code value and its corresponding frequency is incremented.

J. Question 10: Check Whether a String Contains Only Digits

Write a C program to determine whether a string contains only digits.

For example:

- 123456 → Valid
- 123A56 → Invalid
- 12 34 → Invalid

Do not use `isdigit()`.

```

1  #include <stdio.h>
2
3  int main() {
4      char str[200];
5      int i;
6      int onlyDigits = 1;
7
8      printf("Enter a string: ");
9      fgets(str, sizeof(str), stdin);
10
11     for (i = 0; str[i] != '\0'; i++) {
12
13         if (str[i] == '\n')
14             break;
15
16         if (str[i] < '0' || str[i] > '9') {
17             onlyDigits = 0;
18             break;
19         }
20     }
21
22     if (onlyDigits)
23         printf("The string contains only digits.\n");
24     else
25         printf("The string does not contain only digits.\n");
26
27     return 0;
28 }

```

The condition

```
str[i] < '0' || str[i] > '9'
```

identifies any character that is outside the digit range '0' to '9'.

VI. ADDITIONAL LOGICAL CHALLENGES

Try solving the following problems without using predefined string manipulation functions:

- 1) Find the second most frequent character in a string.
- 2) Find the first character that appears exactly once.
- 3) Find the first character that appears more than once.
- 4) Remove all occurrences of a given character from a string.
- 5) Reverse each word of a sentence while keeping the word order unchanged.
- 6) Reverse the order of words in a sentence.
- 7) Check whether a sentence is a palindrome after ignoring spaces and letter case.
- 8) Find the longest sequence of consecutive identical characters.
- 9) Find the character with the maximum frequency.
- 10) Sort the characters of a string in ascending order without using a predefined sorting function.

These problems are intended to strengthen understanding of character arrays, indexing, loops, nested loops, ASCII character codes, and logical state variables before introducing more advanced string-library functions.

VII. KEY TAKEAWAYS

- A C string is terminated by `\0`.
- `scanf("%s", str)` reads a single word.
- `fgets()` can read a complete line containing spaces.
- `getchar()` reads one character at a time.
- `fgets()` may store the newline character `\n`.
- String problems can often be solved using character comparison and loops without library functions.
- `gets()` should not be used because it does not provide bounds checking.

END OF NOTES